

AG35-CET&AG35-EUT

QuecOpen(SDK)设备树开发指导

LTE Standard 模块系列

版本：1.0

日期：2025-04-22

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2025.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-11-23	Jensen ZHANG	文档创建
1.0.0	2023-11-23	Jensen ZHANG	临时版本
1.0.1	2024-12-06	Gabriel LI	临时版本 1. 基于 QuecOpen 方案统一命名，更新文档名称。 2. 新增适用模块 AG35-EUT。
1.0	2025-04-22	Gabriel LI	受控版本

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
2 设备树基本概念	7
2.1. 概述	7
2.2. 设备树文件	7
2.3. 设备树文件之间的关系	8
3 设备树操作	9
3.1. DTS 文件	9
3.1.1. 查询当前使用的 DTS 文件	9
3.1.2. DTS 文件说明	10
3.2. DTSI 文件	11
3.2.1. 查询当前使用的 DTSI 文件	11
3.2.2. DTSI 文件说明	12
3.3. DTB 文件	12
3.4. 新增自定义字段	13
4 附录 参考文档与术语缩写	16

表格索引

表 1: 参考文档	16
表 2: 术语缩写	16

Quectel
Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen[®]方案。QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍设备树相关的基本概念和设备树文件之间的关系，以及如何查看与驱动相关的设备信息文件配置，以及如何查找相关字段解释等，从而实现简易快速的开发。

2 设备树基本概念

2.1. 概述

设备树是 Linux 开发中用于描述硬件信息的文件，包括 CPU 的数量和类别、内存基地址和大小、总线和桥、外设连接、中断控制器和中断使用情况、GPIO 控制器和 GPIO 使用情况、Clock 控制器和 Clock 使用情况。

设备树主要优势在于对于同一 SoC 的不同主板，只需更换设备树文件即可实现不同主板的无差异支持，而无需更换内核文件。

2.2. 设备树文件

1. DTS (Device Tree Source)

ASCII 文本形式的文件，后缀为.dts，用于描述硬件信息，一般存放在内核的/arch/arm/boot/dts/目录下。通常一个 DTS 文件对应一个 ARM 的处理器。详见第 3.1 章。

2. DTSI (Device Tree Source Include)

文本形式的设备树源包含文件，后缀为.dtsi。一个 SoC 可能有多个不同的电路板，而每个电路板拥有一个 DTS 文件，这些 DTS 文件存在许多共同部分。为了减少冗余代码，设备树提取文件中共同部分并保存在 DTSI 文件中，供不同的 DTS 文件共同使用。详见第 3.2 章。

3. DTB (Device Tree Blob)

二进制形式的文件，后缀为.dtb。通过 DTC 编译 DTS 文件生成的二进制文件。Bootloader 在引导内核时，会预先读取 DTB 文件到内存，进而由内核解析。

4. DTC (Device Tree Compiler)

编译工具，用于将.dts 文件编译成.dtb 文件。

2.3. 设备树文件之间的关系

DTS 和 DTSI 文件为源文件，通过 DTC 工具编译为 DTB 文件（二进制），最后内核编译完再以追加的方式打包到 **zImage** 中。三者之间关系如下所示：

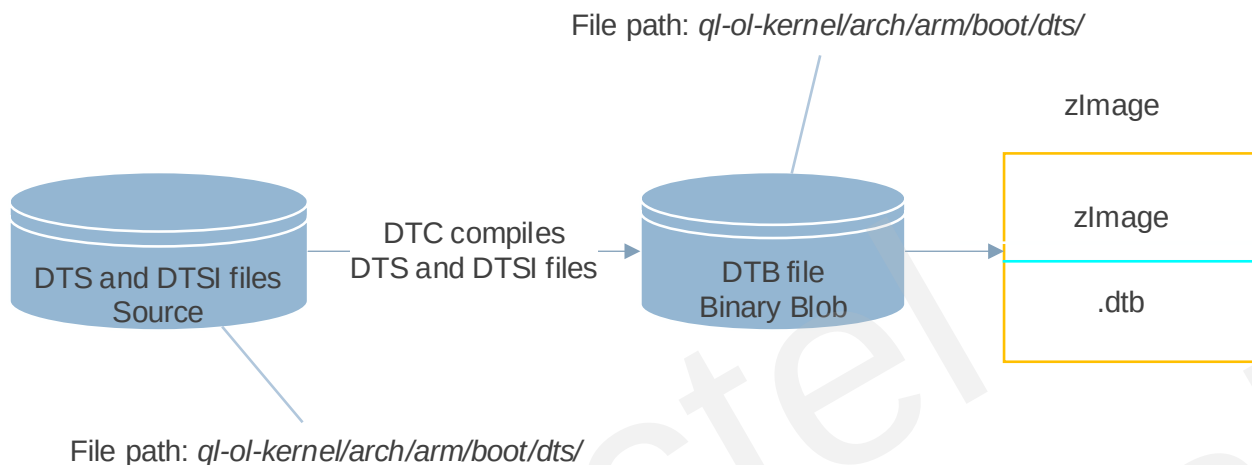


图 1：设备树文件之间的关系

3 设备树操作

在驱动开发阶段，需进行设备树的修改。本章主要介绍如何在模块中查找设备树相关文件以及如何新增自定义字段等。

3.1. DTS 文件

3.1.1. 查询当前使用的 DTS 文件

模块中只有一个 DTS 文件。随着模块固件的不断更新，DTS 文件可能也会相应更改。本节主要介绍如何查询当前使用的 DTS 文件。

由于内核中未配置 `/sys/firmware/devicetree` 节点，用户首先需通过 SDK 中 `Makefile` 中的内核配置来确定 DTS 文件。具体查询步骤如下：

步骤1： 打开 `Makefile` 文件，找到 DTB 文件为 `ql-1806e-std.dtb`。

```
kernel: pre_build
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
  @ [ -f $(Q1_KERNEL_DTB) ] || cp -f $(Q1_KERNEL_DTB) deconfig_asr1806p301 $(Q1_KERNEL_DTB) .config
```

步骤2： DTB 为编译后的文件，则可确定 DTS 文件为 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-std.dts`。

```
1 // SPDX-License-Identifier: GPL-2.0
2 /*
3  * Copyright (C) 2023 ASR Microelectronics Co., Ltd.
4  */
5
6 /dts-v1/;
7 #include "asr1806.dtsi"
8 #include "ql-1806e-common.dtsi"
9 #include "asr_pm802.dtsi"
10 #include "ql-1806e-flash-layout.dtsi"
11
12 /* Motorcomm YT8010A 100Base-T1 PHY RMII */
13 #define CONFIG_USE_YT8010A 0
14
15 /* For 1 to support emmc(default), for 0 to support SD card */
16 #define CONFIG_USE_EMMC 1
17 #define CONFIG_USE_EMMC_SD_ADAPTIVE 0
18
19 /* Config codecs */
20 #define CONFIG_USE_TLV320AIC3104 1
21 #define CONFIG_USE_NAU8810 0
22 #define CONFIG_USE_RT5616 0
23 #define CONFIG_USE_ES8311 0
24
25 #define CONFIG_USE_FCS950U 0
26
27 /* Main entry of device tree */
28 {
29     model = "AG35CET (ASR 1806) Module";
30     compatible = "asr,1803-evb", "asr,1803";
31 };
32
```

3.1.2. DTS 文件说明

DTS 文件中包含一些与模块相关的属性。本节介绍如何查询这些属性的含义。

QuecOpen SDK 的 `ql-ol-kernel/Documentation/devicetree/` 目录下提供了内核文档，详细介绍了 DTS 文件的相关知识。模块的 DTS 文件名称为 `ql-1806e-std.dts`，文件部分内容如下。

```
1 // SPDX-License-Identifier: GPL-2.0
2 /*
3  * Copyright (C) 2023 ASR Microelectronics Co., Ltd.
4  */
5
6 /dts-v1/;
7 #include "asr1806.dtsi"
8 #include "ql-1806e-common.dtsi"
9 #include "asr_pm802.dtsi"
10 #include "ql-1806e-flash-layout.dtsi"
11
12 /* Motorcomm YT8010A 100Base-T1 PHY RMII */
13 #define CONFIG_USE_YT8010A 0
14
15 /* For 1 to support emmc(default), for 0 to support SD card */
16 #define CONFIG_USE_EMMC 1
17 #define CONFIG_USE_EMMC_SD_ADAPTIVE 0
18
19 /* Config codecs */
20 #define CONFIG_USE_TLV320AIC3104 1
21 #define CONFIG_USE_NAU8810 0
22 #define CONFIG_USE_RT5616 0
23 #define CONFIG_USE_ES8311 0
24
25 #define CONFIG_USE_FCS950U 0
26
27 /* Main entry of device tree */
28 / {
29     model = "AG35CET (ASR 1806) Module";
30     compatible = "asr,1803-evb", "asr,1803";
31 };
32
```

`ql-1806e-std.dts` 文件中引用了 `ql-1806-common.dtsi`，文件部分内容如下：

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (C) 2023 Quectel
 */
/ {
    chosen {
        bootargs = "root=/dev/mtdblock5 rootfstype=squashfs init=/etc/preinit noinitrd console=ttyS0,115200 mem=128M";
    };

    aliases {
        serial2 = &uart3;
        serial3 = &uart4;
    };

    firmware {
        optee {
            compatible = "linaro,optee-tz";
            method = "smc";
        };
    };

    memory {
        reg = <0x00000000 0x10000000>;
    };
}
```

从上图中可查看到四个属性，分别是：*model*、*compatible*、*chosen*、*memory*。其中“/”表示根节点。

- *model*
模块 ID。字符串类型。描述模块或芯片型号。
- *compatible*
用来查找节点的方式之一，通常一个驱动对应一个 *compatible* 属性。
- *chosen*
启动参数，作为命令行信息。
- *memory*
内存信息。

备注

以上参数的介绍仅用于说明如何查看属性含义，文件内容不可修改，否则可能会出现无法开机等问题。

3.2. DTSI 文件

3.2.1. 查询当前使用的 DTSI 文件

随着模块固件的不断更新，DTSI 文件也会相应的更改。在实际开发过程中，DTSI 文件可以被多个 DTS 文件包含，并且 DTSI 文件还可以包含其他 DTSI 文件。本章节介绍如何查询当前使用的 DTSI 文件。

根据第 3.1.1 章找到当前使用的 DTS 文件后，在 *ql-ol-kernel/arch/arm/boot/dts* 路径下执行 *la | grep ql-1806e-std* 可找到多个文件：

```
dawn@ubuntu:~/ASR/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-ol-kernel/arch/arm/boot/dts$ la | grep ql-1806e-std
ql-1806e-std.dtb
.ql-1806e-std.dtb.cmd
.ql-1806e-std.dtb.d.dtc.tmp
.ql-1806e-std.dtb.d.pre.tmp
.ql-1806e-std.dtb.dts.tmp
ql-1806e-std.dts
```

可通过查看文件获取 DTSI 文件。以 *ql-1806e-std.dtb.d.pre.tmp* 为例：

```
dawn@ubuntu:~/ASR/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-ol-kernel/arch/arm/boot/dts$ cat .ql-1806e-std.dtb.d.pre.tmp
ql-1806e-std.o: arch/arm/boot/dts/ql-1806e-std.dts \
arch/arm/boot/dts/asr1806.dtsi arch/arm/boot/dts/asr18xx-pinctrl.h \
scripts/dtc/include-prefixes/dt-bindings/power/asr-pm.h \
scripts/dtc/include-prefixes/dt-bindings/clock/asr,asr1803.h \
scripts/dtc/include-prefixes/dt-bindings/clock/timer-mmp.h \
scripts/dtc/include-prefixes/dt-bindings/mmc/asr_sdhci.h \
scripts/dtc/include-prefixes/dt-bindings/phy/phy.h \
include/generated/autoconf.h arch/arm/boot/dts/ql-1806e-common.dtsi \
arch/arm/boot/dts/asr_pm802.dtsi \
arch/arm/boot/dts/ql-1806e-flash-layout.dtsi
```

3.2.2. DTSI 文件说明

DTSI 文件较多，文件涉及的属性也较多。用户可在 `ql-ol-kernel/Documentation/devicetree/bindings/` 路径下，使用 **grep** 搜索相关的字段。

3.3. DTB 文件

在设备树编译流程中，`ql-1806e-std.dtb` 优先级最高，在 DTS 文件层级中最后加载。建议开发时保持 `asr1806.dtsi`、`ql-1806e-common.dtsi`、`asr_pm802.dtsi` 和 `ql-1806e-flash-layout.dtsi` 文件原始配置不变，通过修改 `ql-1806e-std.dtb` 文件来更改配置。相关属性配置参考第 3.1.2 章。示例如下：

- 修改节点对应边缘检测 GPIO 引脚

步骤1： 在 `ql-1806e-common.dtsi` 中看到 `usim1` 节点的 `edge-detect-gpio` 的 PIN 值为 119。

```
988     usim1: usim {
989         compatible = "asr,usim1";
990         pinctrl-names = "default", "sleep";
991         pinctrl-0 = <&usim1_pmx_func>;
992         pinctrl-1 = <&usim1_pmx_func_sleep>;
993         edge-detect-gpio = <119>; /* GPIO19: SIM detect pin */
994         status = "okay";
995     };
```

步骤2： 在 `ql-1806e-std.dtb` 中修改 `usim1` 节点下 `edge-detect-gpio` 的 PIN 值为 19。

```
127 &usim1 {
128     edge-detect-gpio = <19>;
129 };
```

● 关闭模块某项功能

在 `ql-1806e-std.dtb` 文件中，对应功能节点下设置 `status = "disabled"`即可关闭对应的功能。示例如下：

```
1025     quec,gpio_lpm{                                     //gabriel20230517:wakeup_driver,
1026         compatible = "quec,ql_lpm";
1027         pinctrl-names = "default","sleep";
1028         pinctrl-0 = <&wakeup_in_pin &sleep_sys_ind>;
1029         pinctrl-1 = <&wakeup_in_pin_sleep>;
1030         gpios = <&gpio 118 0>,
1031                 <&gpio 38 0>;
1032         ql,gpio-names = "wakeup_in","sleep_sys_ind";
1033         ql,sleep-sys-ind-enable;
1034         ql,sleep-sys-ind-state = <0>;
1035         status = "ok";
1036     };
```

→ change status = "ok"; to status = "disabled";

3.4. 新增自定义字段

以配置 GPIO 为例，介绍如何自定义一个字段并在驱动中解析。

步骤1: 参考 `ql-ol-kernel/arch/arm/boot/dts/asr18xx-pinfunc.h` 配置 GPIO 引脚，如下所示：

```
#define GPIO00 0x0DC
#define GPIO01 0x0E0
#define GPIO02 0x0E4
#define GPIO03 0x0E8
#define GPIO04 0x0EC
#define GPIO05 0x0F0
#define GPIO06 0x0F4
#define GPIO07 0x0F8
#define GPIO08 0x0FC
#define GPIO09 0x100
#define GPIO10 0x104
#define GPIO11 0x108
#define GPIO12 0x10C
#define GPIO13 0x110
#define GPIO14 0x114
#define GPIO15 0x118
#define GPIO16 0x11C
#define GPIO17 0x120
#define GPIO18 0x124
#define GPIO19 0x128
#define GPIO20 0x12C
#define GPIO21 0x130
#define GPIO22 0x134
#define GPIO23 0x138
#define GPIO24 0x13C

#define GPIO25 0x140
#define GPIO26 0x144
#define GPIO27 0x148
#define GPIO28 0x14c
#define GPIO29 0x150
#define GPIO30 0x154
#define GPIO31 0x158
#define GPIO32 0x15c
#define GPIO33 0x160
```

步骤2: 修改 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-std.dtb` 文件，配置 `temp_gpio_default` 如下：

```
temp_gpio_default { //自定义一个 temp_gpio_default 节点
    pinctrl-single,pins = <
        GPIO25 AF0 /* gpio25 */
    >;
    DS_MEDIUM;PULL_UP;EDGE_BOTH;LPM_NONE;
}
```

部分 GPIO 属性如下所示：

- 功能选择 GPIO 可复用的功能：AF0~AF7
- 驱动能力 DS_SLOW0; DS_SLOW1; DS_MEDIUM; DS_FAST
- 上下拉选择 PULL_NONE; PULL_UP; PULL_DOWN; PULL_BOTH; PULL_FLOAT
- 边缘检测 EDGE_NONE; EDGE_RISE; EDGE_FALL; EDGE_BOTH
- 低功耗状态 LPM_NONE; LPM_DRIVE_LOW; LPM_DRIVE_HIGH; LPM_FLOAT

步骤3: 修改 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-std.dtb` 文件。

```
xxx{
    compatible = "xxx";           /* 匹配用户 driver */
    pinctrl-names = "temp_default"; /* 定义 pinctrl name, 驱动中使用
                                     pinctrl_lookup_state()接口解析 */
    pinctrl-0 = <& temp_gpio_default>; /* 选中上面定义的 gpio 配置 */
    status = "ok";                 /* 使能此设备节点 */
}
```

步骤4: 设备树配置完成后，在对应的驱动中解析并使能以上配置，相关内核 API 如下：

1) 获取一个 pinctrl 句柄，参数 `dev` 为包含此引脚的 `device` 结构体：

```
/**
 * struct devm_pinctrl_get() - Resource managed pinctrl_get()
 * @dev: the device to obtain the handle for
 *
 * If there is a need to explicitly destroy the returned struct pinctrl,
 * devm_pinctrl_put() should be used, rather than plain pinctrl_put().
 */
struct pinctrl *devm_pinctrl_get(struct device *dev)
```

2) 获取此引脚对应的状态：

```
/**
 * pinctrl_lookup_state() - retrieves a state handle from a pinctrl handle
 * @p: the pinctrl handle to retrieve the state from
 * @name: the state name to retrieve
 */
struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name)
```

3) 设置此引脚为指定状态（设置为 *on* 即表示使能上述配置）：

```
/**
 * pinctrl_select_state() - select/activate/program a pinctrl state to HW
 * @p: the pinctrl handle for the device that requests configuration
 * @state: the state handle to select/activate/program
 */
int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *state)
```


4 附录 参考文档与术语缩写

表 1: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 2: 术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序编程接口
CPU	Central Processing Unit	中央处理器
DTB	Device Tree Blob	设备树二进制文件
DTC	Device Tree Compiler	设备树编译器
DTS	Device Tree Source	设备树源文件
DTSI	Device Tree Source Include	包含设备树源文件
GPIO	General-Purpose Input/Output	通用型输入/输出
ID	Identifier	标识符
IoV	Internet of Vehicles	车联网
LPM	Low Power Mode	低功耗模式
SDK	Software Development Kit	软件开发工具包
SoC	System on Chip	芯片系统